

R B
Payment Authorization
P2: Thread Blocking

CompletableFuture Orchestration Graphs

Model async workflows as graphs. Chain, combine, and compose.

1. WHAT IS IT?

CompletableFuture can build a DAG Directed Acyclic Graph of asynchronous steps.
Nodes = stages (sync actions or transforms)
Edges = asynchronous (stepped) edges (waiter-waiter)
Data flows through the graph from flow to flow.

COMMON BUILDING BLOCKS

- `thenApply(Fn)` Transform result
- `thenCompose(BiFunction, fn)` Compose result
- `thenCombine(BiFunction, fn)` Combine result
- `thenAccept(Fn)` Handle success / failure
- `thenAcceptBoth(BiFunction, fn)` Handle success / failure
- `thenAcceptEither(BiFunction, fn)` Handle success / failure
- `thenComposeAsync(BiFunction, fn)` Compose result
- `thenCombineAsync(BiFunction, fn)` Combine result
- `thenAcceptAsync(Fn)` Handle success / failure
- `thenAcceptBothAsync(BiFunction, fn)` Handle success / failure
- `thenAcceptEitherAsync(BiFunction, fn)` Handle success / failure

LEGEND

- Green circle: Completed Step
- Blue circle: Async Step (in progress)
- Red circle: Failed Step
- Yellow circle: Suspended / Idle Step
- Black circle: Error Step

COMMON ORCHESTRATION PATTERNS

- 1. SEQUENTIAL CHAIN**
Run tasks one after another.
`A -> B -> C -> D`
`x.thenCompose(y).thenCompose(z).thenCompose(w)`
- 2. PARALLEL TASK-OUT (fork)**
Run tasks in parallel, wait for all.
`A -> B, C, D`
`A.thenCompose(B, C, D)`
- 3. FAN-OUT / FAN-IN (thenCombine)**
Run tasks in parallel, combine results.
`A -> B, C, D`
`A.thenCombine(B, C, D)`
- 4. DEPENDENT TASK-OUT**
After A completes, run B and C in parallel.
`A -> B, C`
`A.thenCompose(B, C)`
- 5. PIPELINE WITH TRANSFORMS**
Transform results at each step.
`A -> B -> C -> D`
`A.thenApply(B).thenApply(C).thenApply(D)`
- 6. ERROR HANDLING**
Recover from errors in the pipeline.
`A -> B -> C`
`A.thenCompose(B).thenCompose(C).thenCompose(D)`
- 7. FIRST COMPLETED (anyOf)**
Use the result of whichever completes first.
`A -> B, C`
`A.anyOf(B, C)`
- 8. TIMEOUT PATTERNS**
Fail back task takes too long.
`A -> B`
`A.withTimeout(B, timeout)`

EXAMPLE: COMPLEX WORKFLOW

Example of a complex workflow using CompletableFuture and its building blocks.

TIPS

- 1. Plan the workflow carefully, considering dependencies and data flow.
- 2. Use `thenCompose` for sequential steps and `thenCombine` for parallel steps.
- 3. Use `anyOf` to wait for the first of multiple tasks to complete.
- 4. Use `withTimeout` to prevent tasks from running indefinitely.
- 5. Use `thenAccept` for handling success or failure.
- 6. Use `thenAcceptBoth` for handling multiple results.
- 7. Use `thenAcceptEither` for handling multiple results.
- 8. Use `thenComposeAsync` for asynchronous composition.
- 9. Use `thenCombineAsync` for asynchronous combination.
- 10. Use `thenAcceptAsync` for asynchronous handling.

BEST PRACTICES

- 1. Model your workflow as a graph first.
- 2. Choose the right building blocks for the job.
- 3. Handle exceptions close to the source.
- 4. Avoid blocking calls in async steps.
- 5. Use timeouts to prevent tasks from running indefinitely.
- 6. Log and monitor key boundaries.

Backpressure Modeling

Model, detect, and control pressure in data pipelines to keep the system stable.

1. WHAT IS BACKPRESSURE?

Backpressure is a feedback signal from downstream to upstream when the system can't keep up with the incoming load.

2. HOW IT WORKS (FEEDBACK LOOP)

Monitor system health, detect pressure, adjust rate, signal upstream.

3. MODEL THE SYSTEM

Queueing model (example: M/M/1). Key Utilization Metrics: $\rho = \lambda / \mu$, $\rho < 1$ (stable), $\rho \geq 1$ (unstable).

4. PRESSURE INDICATORS

Indicator: Queue length, Delay, Error rate, Throughput. What to look for: Growing queue, Rising latency, Rising error rate, Throughput drop.

5. BACKPRESSURE STRATEGIES (CHOOSE BASED ON YOUR PIPELINE)

- 5.1 Rate Limiting**: Cap the rate of incoming requests.
- 5.2 Circuit Breaker**: Stop the pipeline if it fails too often.
- 5.3 Bounded Queue**: Limit the number of items in the queue.
- 5.4 Load Shedding / Dropping**: Drop items when the system is overloaded.
- 5.5 Adaptive Control**: Dynamically adjust the feedback loop.

6. STEP-BY-STEP MODELING PROCESS

1. Define the system. 2. Model the system. 3. Simulate the system. 4. Analyze the results. 5. Optimize the system. 6. Deploy the system.

7. EXAMPLE: END-TO-END FLOW

Flow from input to output, showing the backpressure signal.

8. BEST PRACTICES

- 1. Measure everything: latency, throughput, error rate.
- 2. Model the system first.
- 3. Keep queues bounded.
- 4. Drop gracefully: no data loss, no corruption.
- 5. Keep backpressure close to the source.
- 6. Test under load: validate, then iterate.
- 7. Continuously monitor and tune.

Lock-Free Collections

High-performance concurrent data structures without locks.

WHAT IS WHY?

Lock-free collections allow multiple threads to access data concurrently without using locks or blocking.

CORE PRINCIPLES

- 1. Atomicity with CAS**: Compare-and-Swap (CAS) is the atomic operation.
- 2. No Blocking**: Threads don't block if they fail, they only retry.
- 3. Linearizability**: Each operation appears to occur at a single point in time.
- 4. Progress Guarantee**: Lock-free system makes progress, not necessarily wait-free.
- 5. ABA Problem**: Value changes from A to B and back to A - CAS will catch it.

COMMON LOCK-FREE COLLECTIONS

- ConcurrentLinkedQueue**: Lock-free queue.
- ConcurrentLinkedDeque**: Lock-free deque.
- CopyOnWriteArrayList**: Thread-safe list for read-only access.
- AtomicReference / Field Operations**: Atomic operations on fields.

HOW LOCK-FREE WORKS (EXAMPLE: STACK PUSH)

Step-by-step process of pushing an element onto a lock-free stack.

IF CAS FAILS?

Handle the failure by retrying the operation.

BEST PRACTICES

- 1. Use the right tool: Choose the collection that matches your access pattern.
- 2. Expect retries: CAS may fail under contention - design for retry loops.
- 3. Mind the ABA problem: Use AtomicStampedReference or versioned atomic types like ABA sensitivity.
- 4. Measure & test under load: Performance depends on contention and workload.
- 5. Keep operations side-effect free: Read-only operations must be safe to re-execute.
- 6. Understand progress guarantees: Lock-free means that some progress is guaranteed.

Idempotent API Design

Same request, same effect - safe to retry.

WHAT IS IDEMPOTENCY?

An idempotent API ensures that making the same request multiple times has the same effect as making it once.

CORE PRINCIPLES

- 1. UNIQUE REQUEST IDENTIFIER**: Every request must have a unique identifier.
- 2. STORE AND CHECK**: Store the request identifier and check for duplicates.
- 3. SAME RESPONSE**: Return the same response for duplicate requests.
- 4. EMPTY & CLEANUP**: Return empty response for duplicate requests.
- 5. SCOPE & CONSTRAINT**: Define the scope and constraints for the API.

DEMPOTENT VS NON-IDEMPOTENT

Comparison of idempotent and non-idempotent API requests.

HOW IDEMPOTENCY WORKS (FLOW)

Flowchart showing the process of handling an idempotent request.

EXAMPLE: CREATE ORDER API

Example of an idempotent API for creating an order.

KEY GUIDELINES

- 1. Plan the API design.
- 2. Use a unique request identifier.
- 3. Store the request identifier.
- 4. Check for duplicates.
- 5. Return the same response.
- 6. Use a unique request identifier.

CHOOSE SCOPE WISELY

Choose the scope of the API based on the use case.

WHAT TO STORE

Store the request identifier and the response.

HANDLING CONFLICTS

Handle conflicts between different requests.

BEST PRACTICES CHECKLIST

- 1. Always use a unique request identifier.
- 2. Store the request identifier.
- 3. Check for duplicates.
- 4. Return the same response.
- 5. Use a unique request identifier.

GOLDEN RULE

The server may process a request at most once per idempotent key.

REMARK

Safe retries. No duplicates. No errors.